

Protocolo de robustez XAUUSD — dossier de diseño e implementación

Qué es este documento. Un *design dossier* en el sentido de `docs/AgentPDFs/README.md` : el plan completo para convertir `Protocolo_robustez_XAUUSD.pdf` (2026-09-20) en código, con lo que ya está construido marcado como tal y lo que falta descrito con el detalle suficiente para que varios agentes lo implementen en paralelo.

Para quién. Para el dueño, y para el agente que recoja cualquiera de los lotes W2 a W8. Los contratos de la §2 son la interfaz entre lotes: se pueden construir módulos contra ellos sin que exista el módulo anterior.

Regla que hereda de esta carpeta. El dossier guarda la narrativa y el plan; `knowhow/` guarda los hechos.

Donde los dos discrepen, manda `knowhow/`. Todo lo marcado ¶ aquí está ya duplicado en `knowhow/01-file-formats.md` y `knowhow/04-export.md`, que es su sitio permanente.

Generado el 2026-09-21. Continúa `Protocolo_robustez_XAUUSD.pdf` y `plataforma-unificada-2026-09-20.md`.

Contexto

El PDF `Protocolo_robustez_XAUUSD.pdf` (2026-09-20) define un protocolo en dos etapas: el SPP de SQX como reconocimiento, y 5.000 variantes `.sqx` fabricadas como experimento. El PDF describe **qué** medir; no describe **cómo fabricar** las 5.000 variantes, ni cómo encadenar todo por estrategia.

El dueño quiere además que esto sea un **pipeline automatizable**: una estrategia madre del databank entra, sale un veredicto, se borra lo innecesario, y empieza la siguiente. Con ~100 estrategias madre eso son días de máquina desatendida, lo que convierte la reanudabilidad y la higiene de disco en requisitos, no en detalles.

Resultado buscado: una **regla de selección de parámetros con su coste medido**, aplicable a la siguiente estrategia y a la siguiente.

Contexto añadido 2026-09-21 — la plataforma unificada

`docs/AgentPDFs/plataforma-unificada-2026-09-20.md` describe hacia dónde va todo esto: un demonio FastAPI con cola unificada de trabajos, una carcasa de escritorio, y SQX invocado headless en vez de abierto a mano. **No se construye ahora** — el dueño quiere los módulos de análisis terminados y perfilados primero, y el propio documento lo respalda (su riesgo 5: *una interfaz bonita sobre conclusiones no reproducibles es peor que una terminal fea*).

Pero seis decisiones de este plan cambian por saberlo, y **cinco son baratas ahora y caras después**. Están marcadas ¶ donde aplican.

Y un requisito duro del dueño, 2026-09-21: la plataforma debe correr en varias máquinas Linux.

Windows se quiere para desarrollo, y hoy solo sirve para la mitad analítica. Eso asciende R4 del documento — `bin/sqx-worker.sh` es bash — de «decidirlo pronto» a **requisito**. Ver §6bis y `docs/SETUP-NEW-`

1 · Arquitectura — seis piezas

#	Pieza	Pregunta que contesta	Ubicación
0	core/surface.py	(librería) la matemática de cualquier rejilla de parámetros	core/
1	strategies/sppUltra/	¿qué mueve el resultado y merece esta estrategia las 5.000 variantes?	strategies/
2	sqx/variants/	fabricar, cargar, correr y recoger las 5.000 variantes	sqx/
3	strategies/walkForwardCorrelation/	¿sobrevive la superficie fuera de muestra, y qué regla usar?	strategies/
4	strategies/walkForwardMatrix/	¿lo que optimiza bien predice lo que va bien después?	strategies/
5	pipeline/	encadenar 1 → 4 por estrategia, reanudable y auditable	raíz

Extensiones a módulos existentes, no módulos nuevos:

- **\$9 multi-mercado** → `strategies/crossmarket/`, que ya construye nulos de ocupación igualada.
- **Higiene de disco** → `perf/disk/`, que ya inventaría `AlgoData` (`inventory.py`, `duplicates.py`, `formats.py`). Le falta un **presupuesto** que falle y una **política de retención**.

Por qué esta partición y no «SPP + WF»

Un módulo por **modo de fallo**, y aquí hay tres distintos:

1. `sppUltra` — leer una rejilla **que no elegiste** como si fuera un censo.
2. `sqx/variants` — **corrupción silenciosa**: renombrado en colisión, `<Fingerprint>` heredado, borrado por sync, tupla ≠ fichero.
3. `walkForwardCorrelation` — confundir **nivel** con **orden**; n efectivo falso.

Además, meter la fabricación dentro de una carpeta de análisis rompería el carril de `sqx/`, que es el dueño de todo lo que escribe o conduce SQX (`sqx/CLAUDE.md`).

2 · Los contratos — lo que permite trabajar en paralelo

Estos cuatro ficheros son la interfaz entre lotes. Se congelan antes de escribir código. Un agente puede construir su módulo contra un fixture que cumpla el contrato sin que exista el módulo anterior.

C1 · `design_brief.json` — sppUltra → sqx/variants

```
{ "strategy": "Strategy 17.9.39", "project": "XAUUSD", "source_databank": "SPP IS",
  "verdict": "proceed|noise", "n_eff": 8412, "noise_max": 1.83, "observed_max": 4.61,
  "parameters": [ { "name": "DIPeriod1", "eta2": 0.184, "inert": false,
    "center": 67, "center_rule": "best_value", "levels": [55,61,67,73,79],
    "original": 64, "argmax_is": 71, "plateau_width": 12, "is_shift": false } ],
  "frozen": [ { "name": "CBlock_SqzMmnInt21", "value": 21, "reason": "exact_duplicates" } ],
  "strata": { "neighbourhood": 0.20, "factorial": 0.60, "coverage": 0.20 }, "n_target": 5000 }
```

C2 · `manifest.parquet` — sqx/variants → todos los análisis

Una fila por variante fabricada. **La clave de unión de todo el estudio.**

columna	qué es
<code>variant_id</code>	P00000 ..., escrito también dentro del .sqx (nombre o comentario)
<code>sqx_name</code>	el nombre con el que SQX lo devuelve, tras posible renombrado en colisión
<code>stratum</code>	<code>neighbourhood</code> <code>factorial</code> <code>coverage</code> <code>canary</code> <code>origin</code>
<code>param_<nombre></code>	una columna por parámetro del diseño
<code>tuple_hash</code>	hash de la tupla, para detectar duplicados de fabricación
<code>origin</code>	<code>true</code> en la estrategia madre — collect.py se niega a borrarla
<code>canary_expect_netprofit</code> , <code>canary_expect_trades</code>	solo en los canarios

C3 · `metrics.parquet` — 41 métricas × {IS, OOS}

`variant_id` + `<métrica>_IS` + `<métrica>_OOS` . Lista cerrada en §3.

C4 · `trades.parquet` — 12 columnas, particionado por bloques de 500 variantes

`variant_id` + `Open time` · `Close time` · `Type` · `Open price` · `Close price` · `Size` · `Profit/Loss` · `Balance` · `MAE ($)` · `MFE ($)` · `Sample type` · `Close type`

El orden de fila ES el `Ticket` . Nunca reordenar al escribir.

C5 · `state.json` — el ledger del pipeline

▮ **Se escribe DURANTE cada etapa, no al terminarla.** Es la corrección más importante que trae el documento de plataforma. El demonio necesita progreso en streaming para su barra; si cada etapa va anotando avance mientras corre (`stages.<nombre>.progress` , 0..100, más una línea de estado), el demonio lo sirve sin tocar nada. Si solo se escribe al final, hay que instrumentar las cinco etapas otra vez. Ruta B del documento — `tail` del log de `ProgressEngine` — es de dónde sale el porcentaje para las etapas que corren dentro de SQX.

AlgoData/pipeline/<proyecto>/<estrategia>/state.json . Pesa kilobytes y **sobrevive al borrado**, que es lo que hace el borrado auditable.

```
{ "strategy": "...", "stage": "collected", "stages": {
  "sppultra": {"done_at": "...", "verdict": "proceed", "brief_hash": "..."},
  "built":    {"done_at": "...", "n": 5000, "bytes": 56500000},
  "ran":      {"done_at": "...", "n_returned": 5000, "databank_before": 5000,
               "databank_after": 5000, "canaries": "pass"},
  "collected":{"done_at": "...", "files": [{"path":"...","bytes":...,"sha256":"..."}],
               "deleted": [{"path":"...","bytes":...,"at":"..."}] },
  "costs_provisional": true }
```

3 · Hechos medidos que el código debe respetar

Todos verificados en esta sesión contra AlgoData/raw/XAUUSD/SPP_IS/2026-09-10/ . **Ningún agente necesita volver a medirlos.** Van a knowhow/ en el mismo lote que los usa (regla permanente).

Formato — el hallazgo principal

	tamaño	factor
permutations.csv , 154 cols (hoy)	39,67 MB	1×
Parquet zstd, 154 cols	5,72 MB	6,9×
Parquet zstd, 29 cols curadas	1,51 MB	26×

El formato da 6,9× sin perder un dato; la poda de columnas da 3,8× más y es donde vive el arrepentimiento. Igual en trades: tipar+comprimir da 3,2×, podar 5 columnas solo un 1,4× más. → knowhow/04-export.md

Columnas del SPP

- 21.205 permutaciones × 152 métricas. **42 constantes** (4 AddMarkets*Median , BestWF , EdgeDecayRatio , Parameters , SlopeRatio , TotalData* , y **34 stat:f:NN sin nombre**), **110 varían, 85 con información única** ($|p|>0,999$ dentro de una estrategia).
- **La redundancia es intra-ventana.** NetProfit $\equiv$ CAGR $\equiv$ AnnualPctReturn con $p = 1,000$ dentro de una ventana; **entre ventanas de distinta longitud dejan de serlo** — que es exactamente el sesgo $\sqrt{T}$ del §4a del PDF. Por eso se guardan las dos.
- **Dos columnas llevan un ? literal en el nombre:** CalmarRatio? y AnnualPctReturnDDRatio? . Pedirlas sin el ? da una columna entera de NaN sin error.
- **RExpectancy lleva centinelas:** 99999.0 (3 filas) y -1.0 (15 filas), todas con ~1 operación. Son el 0,08 % — y **ganan el argmax sobre 5.000 variantes.** Filtrar $|v| > 100$ antes de cualquier ranking, en core/surface.py .

Fórmulas verificadas

- **CalmarRatio?** = **CAGR / DrawdownPct** — verificado sobre 20.554 filas; el error mediano de 1,9 % es el redondeo a 2 decimales ($2,20/5,52 = 0,3986 \rightarrow$ almacenado 0,40). **Es un Calmar legítimo: se incluye.** Es además **idéntico byte a byte a AnnualPctReturnDDRatio?** (error 0,00e+00 en las 20.554), así que se guarda **una sola copia**.
- **RExpectancyScore** $\approx$ **RExpectancy** $\times \sqrt{n}$ hasta un factor por estrategia (ρ +0,89 a +0,99, factor 2,0–3,6). Es un estadístico tipo t. **No es redundante** con **RExpectancy** (ρ entre ellas +0,856 a +0,976) y es justo lo que el §4c necesita: penaliza las regiones de pocas operaciones.
- **ZScore** = **(R – μ_R + 0,5) / σ_R** — rachas de Wald-Wolfowitz con corrección de continuidad. Ya documentado en `knowhow/01-file-formats.md`.
- **UlcerPerformanceIndex** = **retorno / UlcerIndex** — **sin verificar en esta instalación.** W1 lo reconcilia con el método del oráculo antes de usarlo; si no reconcilia, se marca análogo declarado.

Trades

- **Ticket** = **orden por (Open time, Close time)** — verificado en las 5 estrategias, 4.115 operaciones: ya vienen ordenadas, 0 **Open time** duplicados, 0 solapes. Parquet preserva el orden de fila, así que **el orden del fichero es el ticket**. $\rightarrow$ `collect.py` comprueba por fichero: ordenado $\wedge$ sin duplicados $\wedge$ sin solapes. Si falla, **conserva Ticket** en ese fichero y lo anota en el manifiesto. (Una estrategia con pirámide rompería el supuesto.)
- **Balance** = **100.000 + cumsum(P/L)**, desviación máxima 0,17 en 763 filas — redondeo, no una comisión aparte. **Se guarda igualmente** por decisión del dueño.
- **Fuera:** **Ticket**, **Symbol** (constante $\rightarrow$ manifiesto), **Time in trade** (= Close – Open, y como texto), **Comment** (763/763 nulos).
- **Close type** **no es prescindible:** **Exit Signal** 662 / **Exit After X Bars** 83 / **End Of Friday** 18. Cuál dispara cambia con los parámetros, que es lo que el estudio mide.

La vista `.vw`

`~/Desktop/SQX/user/settings/views/databanks/Export Data View.vw` existe: **22 columnas IS (sampleType="10") + 13 OOS (sampleType="20")**. Le faltan en OOS **NumberOfTrades** (que el §4c exige en ambas ventanas), **Drawdown**, **Stability**, **RSquared**.

$\rightarrow$ **No se toca.** W3 crea `WFC Variants.vw` en el worker, **simétrica:** las mismas 41 en IS y en OOS.

$\triangle$ `knowhow/08-columns.md` : el valor de una columna se congela en el `.sqx` al calcular el resultado. **Una columna añadida a la vista después sale 0 en toda estrategia anterior, sin aviso y sin celda vacía.** La vista tiene que estar cerrada **antes** de lanzar el retest de las 5.000.

Coste (del PDF, log del master, 95 núcleos)

SPP por estrategia	~74 s
Retest	~24 estrategias/s $\rightarrow$ 5.000 $\approx$ 3,5 min
<code>.sqx</code> 5 miembros / sin <code>optimizationProfile</code> / íntegro	11,3 KB / 100 KB / 5.215 KB

Pico de disco con el bucle una-estrategia-cada-vez: **57 MB** (mejor caso) a **500 MB** (peor). No es un problema acumulado. Trades: **~200 MB por estrategia madre y mercado**, ~20 GB por 100 madres.

4 · La lista de métricas — cerrada, 41 columnas

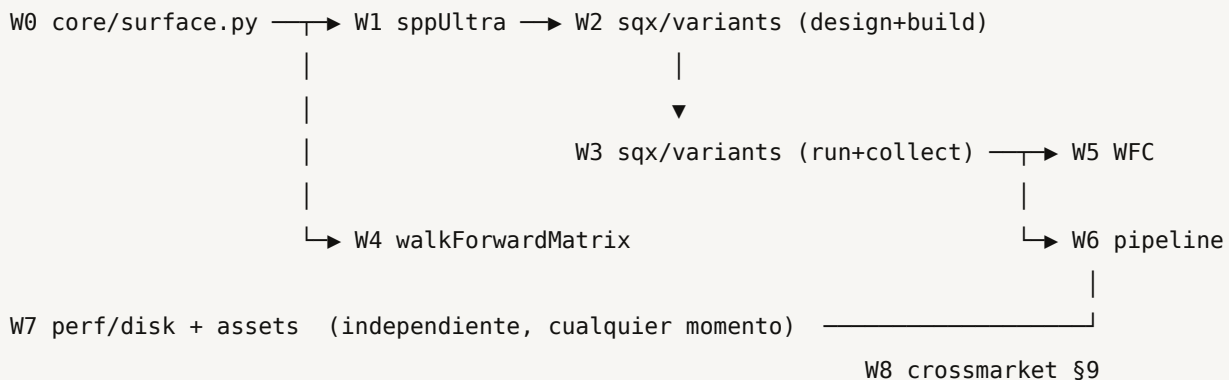
Las mismas en IS y en OOS. Δ = no reconstruible en Python (equity diaria); si no se exporta, se pierde.

- **Resultado (4)** — NetProfit · CAGR · GrossProfit · GrossLoss
- **Riesgo (4)** — Drawdown · DrawdownPct · StandardDev · UlcerIndex Δ
- **Ratios (13)** — SharpeRatio Δ · SortinoRatio Δ · ProbSharpeRatio Δ · ProfitFactor · ReturnDDRatio · CalmarRatio? · RecoveryFactor · SQN · SQNScore · RExpectancy · RExpectancyScore · TRLRatio · UlcerPerformanceIndex Δ
- **Operaciones (9)** — NumberOfTrades · NumberOfProfits · NumberOfLosses · WinningPct · AvgTrade · AvgWin · AvgLoss · MaxLoss · PayoutRatio
- **Rachas (3)** — MaxConsecWins · MaxConsecLosses · ZScore
- **Tiempo (4)** — Exposure · AvgBarsInTrade · Stagnation · TotalTradingYears
- **Calidad (4)** — RSquared Δ · Stability Δ · Efficiency · AmbiguousTrades

Excluidas por decisión del dueño: Fitness, KellyFormula, Outlier, AnnualPctReturnDDRatio? (duplicado exacto de CalmarRatio?), DoFRatio ($p = 1,000$ con NumberOfTrades), ParameterCount, AnnualPctReturn.

Exposure, TotalTradingYears, NumberOfProfits/Losses, StandardDev y AmbiguousTrades entran porque el protocolo no funciona sin ellas: ocupación para el §9, longitud de ventana para el §4a, umbral por año para el §4c, $ddof=0$ para cualquier análogo de Sharpe, y artefactos del motor.

5 · Lotes de trabajo y grafo de dependencias



Paralelizable desde el minuto uno, contra los contratos de §2 y fixtures:

lote	puede empezar	agente
W0 core/surface.py	ya	A
W7 perf/disk + assets	ya	B
W4 walkForwardMatrix	ya — el export ya existe	C
W1 sppUltra	tras W0 (o contra un stub de surface)	A
W2 variants diseño+fabricación	contra un design_brief.json fixture	D
W6 pipeline	contra los contratos; no necesita ningún módulo hecho	E
W3 variants ejecución	bloqueado por costes + worker	D
W5 walkForwardCorrelation	contra fixtures de C3/C4	F
W8 crossmarket \$9	tras W3 × M mercados	—

Hasta **cinco agentes en paralelo** (A, B, C, D, E) sin pisarse: tocan carpetas disjuntas.

6 · Detalle por lote

W0 · core/surface.py — la librería compartida

Es el gridshift.py del §11 del PDF, pero en core/ porque lo importan dos estudios desde el día uno y tasks/ es nivel-población. Precedente: core/significance.py .

Por rule 1 de CODESTYLE (250 líneas) probablemente sea core/surface/ con 3 ficheros:

fichero	contenido
dedupe.py	dedupe por (NetProfit , NumberOfTrades), n_eff , filtro de centinelas, bootstrap por tupla/clúster — nunca por fila
shift.py	Hodges-Lehmann con IC · δ de Cliff · curva QQ y exceso_cola · ratio de dispersión ajustado por $\sqrt{(n_{IS}/n_{OOS})}$
plateau.py	área de meseta · máximo esperado bajo el nulo $\sigma \cdot \sqrt{(2 \cdot \ln n_{eff})}$ · DSR

Tests golden en tests/ con rejillas sintéticas de propiedad conocida: - superficie plana → área de meseta $\approx 0,5$, HL-shift ≈ 0 - superficie **barajada** (mismo histograma, orden aleatorio) → HL-shift ≈ 0 **y $p \approx 0$** — es el caso ciego del §2 del PDF y el que justifica todo el emparejamiento - superficie desplazada en bloque → HL-shift = el desplazamiento, p alto

Reutiliza core/significance.py (BH, suelo de significación) — no reimplementar.

W1 · strategies/sppUltra/

Forma del repo (strategies/retest/): inputs/ model/ verdict/ render/ + run.py + report.py + config.yaml + README.md + POSSIBLE_IMPROVEMENTS.md .

capa	contenido
inputs/	lee los 5 CSV de <code>sqx/export/export_spp.py</code> ; el dueño nombra los databanks
model/	η^2 por parámetro · test de duplicados exactos (agrupar por todo menos uno) · perfil marginal · ancho de meseta · <code>BestValue</code>
verdict/	máximo bajo el nulo con <code>n_eff</code> , DSR · <code>proceed</code> <code>noise</code>
render/	informe + figuras; <code>explorer/</code> opcional después

Dos salidas, no una: el informe, y el `design_brief.json` (C1).

Fases 0/1/2/3 del §3 del PDF: el módulo las lee todas; **las tiradas las lanza el dueño** (regla 3). La Fase 2 (ventanas placebo 2008–2012 y 2013–2017) es la que calibra el exponente $\sqrt{T}$ del §4a.

Reutiliza `core/optprofile.py` y el export existente `sqx/export/export_spp.py` — **no duplicar**.

W2 · `sqx/variants/` — diseño y fabricación (sin tocar SQX)

fichero	qué hace
<code>design.py</code>	<code>design_brief.json</code> → CSV de tuplas. Tres estratos: vecindad ~20 % (rejilla completa a distancia 1 y 2), factorial grueso saturado ~60 %, cobertura global Sobol/LHS ~20 % incluidos los congelados
<code>build.py</code>	CSV → N <code>.sqx</code> + <code>manifest.parquet</code> . Borra <Fingerprint> del <code>settings.xml</code> , escribe <code>variant_id</code> dentro de la estrategia, marca <code>origin: true</code> en la madre
<code>canaries.py</code>	inyecta los 3 canarios de resultado conocido, la tupla original, y pares que difieren solo en un parámetro inerte

Canarios (MEDIDO, del PDF): `P00000` NetProfit 9.076,34 / 844 ops · `P00001` 2.039,86 / 4 ops · `P00002` 3.077,74 / 6 ops.

Diseño A / diseño B: shifts fijos + resto fino (A), y solo shifts 7^k saturable (B). Dos databanks, dos tasks. Un parámetro entra **si mueve el resultado** (η^2 + duplicados), no según si el SPP lo permutó — decisión corregida del dueño, §10 del PDF.

El diseño debe ser simétrico y lo bastante ancho para contener el original y el argmax IS, o una meseta que se movió cae fuera de la rejilla y nadie se entera (§5 del PDF).

Reutiliza `core/sqxfile.py` (lectura/repack de `.sqx`) y `core/manifest.py`.

W3 · `sqx/variants/` — ejecución y recogida [△](#) bloqueado

fichero	qué hace
<code>views.py</code>	genera <code>WFC Variants.vw</code> con las 41 en IS y OOS — antes del retest
<code>run.py</code>	proyecto dedicado (clon de XAUUSD) en el worker 5060 → carga el databank → lanza la task de retest pareada
<code>collect.py</code>	verifica completitud + canarios → exporta C3 y C4 → borra con auditoría

▮ **La guarda contra el borrado por sync sale a `sqx/lifecycle.py`, no vive dentro de `run.py`**. Es a la vez el riesgo nº 1 de este lote y, según el documento de plataforma, «la parte técnicamente delicada del proyecto entero» (su R3). Es la misma mitigación —snapshot + conteo antes/después— y el futuro

SQXAdapter debe heredarla en vez de reimplementar el mecanismo que, si falla, destruye trabajo.

▮ **Éste es el momento de portar `bin/sqx-worker.sh` a Python.** El dueño ha fijado que la plataforma corre en varias máquinas Linux y que Windows se quiere para desarrollo. Todo W3 se apoya en `core/worker.py`, que llama al script bash (`rsync`, `ss`, `curl`, `setsid`) y que hoy obliga a `require_posix()` en ocho puntos de entrada. Es ~1 día y este lote es cuando esa capa se toca de todos modos. Ver §6bis.

La task de retest: un Setup 2008–2022 con `<OutOfSample><Range dateFrom="2018.01.01" .../>` guarda `sampleType` 10 y 20 en el mismo `.sqx`. **Una sola task da las dos muestras** (MEDIDO, §6 del PDF).

Guarda contra la regla 1, obligatoria. Todo sync borra los `.sqx` en disco que no estén en memoria — y el log del master lo tiene con un auto-sync horario, no un cierre: `'Project - USDJPY/WFM' before sync 248 / after sync 36 / removed 248`. → `run.py` **cuenta el databank antes y después** y aborta ruidosamente si la cuenta cae. → Snapshot de `user/projects` antes de cualquier cosa que reinicie SQX.

Dos incógnitas que se matan con 3 ficheros (§6 del PDF, ABIERTO): ¿carga SQX el `.sqx` de 5 miembros sin `orders.bin`? ¿deduplica el databank por el `<Fingerprint>` heredado? El test se valida solo con los canarios. **Esta prueba va primero, antes de fabricar 5.000.**

Export masivo — medir con 100 antes de comprometerse a 5.000 (ABIERTO del §2). Streaming: el CSV por estrategia que suelta SQX se lee y se anexa a Parquet particionado por bloques de ~500; la memoria queda acotada por estrategia, no por databank. Perfilar con `perf` / `perf-profiler`.

Borrado: solo tras verificar completitud + canarios + hashes escritos en el ledger. Nunca la madre (`origin: true`). Por defecto **conserva el databank de la estrategia anterior** mientras corre la siguiente — 500 MB de pico por una red de seguridad de una vuelta entera.

Reutiliza `core/worker.py` (start/stop/wait_ready), `core/exportdrv.py`, `core/cfx.py`, y la skill `sqx-strategy-project` para clonar.

W4 · [strategies/walkForwardMatrix/](#)

La mitad que falta de algo ya construido. `sqx/export/export_wfm.py` + `core/wfmatrix.py` + `core/wftrades.py` + `docs/manual/09-wfm.md` ya sacan la cuadrícula completa. **No existe el análisis.** Por eso es el lote con mejor relación resultado/riesgo y **no depende de nada**.

Contesta: ¿lo que va bien optimizando predice lo que va bien después? Y sostiene el holdout 2022–2026.

⚠ **Puerta de pre-registro.** El §10 del PDF: WFM y WFC leen la misma ventana el mismo día, así que hay que escribir **antes de correr, con fecha**, qué contaría como aprobado. Escrito después no vale. → `reports/XAUUSD/_wfc/<fecha>/PREREGISTRO.md`, commiteado, y **pipeline se niega a entrar en la etapa WFM si no existe con fecha anterior**. Si es un papel opcional, no se escribirá nunca.

Aviso a incluir en el informe: en oro, 2022–2026 son ~4 años de **un solo régimen**. Sobrevivir ahí es una observación, no generalidad.

W5 · [strategies/walkForwardCorrelation/](#)

capa	contenido
<code>model/</code>	las 3 correcciones de sesgo del §4: $\sqrt{T}$ calibrado con las placebo · n efectivo · umbral por año de operaciones, y tabla con y sin filtro + % de rejilla excluido
<code>measure/</code>	ρ de Spearman IS $\leftrightarrow$ OOS global y por distancia al óptimo · deriva del óptimo
<code>verdict/</code>	coste de 3 reglas de selección (argmax, centro de meseta, azar) · CSCV/PBO · descartar/dudar/mantener
<code>render/</code>	curva QQ, área de meseta, perfiles marginales

▮ Sin `explorer/` . Ningún estudio nuevo trae su propio servidor Flask. El documento de plataforma convierte los paneles en **vistas** sobre el demonio, y ya hay tres `serve.py` (~4.700 líneas) que habrá que reescribir. A cambio, **regla nueva para todo estudio: dos salidas, el informe humano y un resultado estructurado**. `sppUltra` ya emite `design_brief.json`; `walkForwardMatrix` emite `cell_correlations.csv` . Las vistas futuras leen eso y no recalculan nada.

CSCV (§8d): matriz N configuraciones $\times$ T periodos desde los trades, $S=10 \rightarrow 252$ particiones, corrido **tres veces** cambiando solo la regla de selección. Con ~60 ops/año hay que agregar a P&L **semanal o mensual** — una matriz diaria sería casi toda ceros.

Por estrategia, nunca agregado. Se agrega después, sobre los estadísticos, con Fisher-z.

Referencia MEDIDA contra la que comparar ρ : la optimización secuencial dio $\rho \approx +0,31$ (IC 95 % $+0,10$ a $+0,54$) — pero es una estrella de ejes, no una superficie.

Umrales del veredicto — PROPUESTOS, pendientes de aprobación (§11 del PDF), en `config.yaml` :

veredicto	condición
Descartar	máximo IS no supera el máximo bajo el nulo, o área de meseta OOS < 50 % de la IS, o exceso de cola > 1 desviación
Dudar	el OOS cae fuera de la envolvente de las placebo pero la meseta aguanta
Mantener	meseta conservada y ρ positivo sobre el nulo y PBO < 50 %

W6 · [pipeline/](#)

fichero	qué hace
<code>ledger.py</code>	lee/escribe <code>state.json</code> (C5); append-only por etapa
<code>stages.py</code>	el registro de etapas: nombre $\rightarrow$ comando $\rightarrow$ puerta de entrada $\rightarrow$ puerta de salida
<code>run.py</code>	envoltorio finito que las llama en orden

▮ `pipeline/` y el **AlgoDaemon son la misma pieza**. La fase 0 del documento de plataforma es «demonio + cola unificada». Si esto se escribe como orquestador de terminal, se reescribe entero. Dos consecuencias de diseño:

- **Las etapas son datos, no código**. `stages.py` expone un registro (nombre $\rightarrow$ comando $\rightarrow$ puerta de entrada $\rightarrow$ puerta de salida) y `run.py` es una CLI fina encima. El demonio importa el mismo registro.

- **La lista de etapas vive en YAML**, porque es literalmente la `receta:` de la §9 del documento de plataforma. El demonio lee el mismo fichero; no hay dos definiciones de qué es un pipeline.

Tres propiedades no negociables: - Cada etapa es su propio `python3 -m ...` con entrada y salida en disco. `run.py` **no contiene lógica propia**: si desapareciera, todo seguiría funcionando a mano. (*Requisito explícito del dueño.*) - **Idempotente y reanudable**. Re-correr una etapa terminada no hace nada; una caída a mitad de las 5.000 arranca donde estaba. - **Puertas duras que se niegan a avanzar**: canarios fallidos, variantes faltantes, cuenta de databank caída, `core.assets` bloqueando, pre-registro ausente.

Modo inicial `--from spp`: el dueño corre los SPP y nombra los databanks. En esta fase el veredicto `noise` **no salta** la estrategia — se fabrica igualmente para tener el dato (decisión del dueño; revisable después).

W7 · Higiene de disco y costes — independiente

perf/disk/ ya existe (`inventory.py`, `duplicates.py`, `formats.py`, `report.py`) y ya escribe `disk.csv`, `duplicates.csv`, `formats.csv`. Lo que falta:

- un **presupuesto** en `perf/config.yaml` (GB por rama de `AlgoData`) que haga **salir non-zero**, igual que `catalogue` ya hace con las regresiones — así puede ir en cron;
- una **política de retención** que cruce `disk.csv` con los `state.json` del pipeline: señalar `raw/` fechados sin ledger vivo, `.sqx` de databanks ya recogidos, y exports huérfanos;
- `perf/disk/report.py` muestra **qué se puede borrar y cuánto libera**, sin borrarlo.

Hoy: `AlgoData` = 2,3 GB, `raw/` = 2,1 GB (3.051 `.csv`, 862 `.sqx`, 90 `.parquet`). **No se convierte nada existente** — Parquet solo de aquí en adelante (decisión del dueño).

assets/XAUUSD.yaml: rellenar `spread.use`, `commission.use`, `point_value.use` con los defaults de SQX y why: "provisional – default de SQX, pendiente de valores reales". El dueño los corregirá. Todo informe generado con costes provisionales lo estampa en su cabecera (`costs_provisional` en C5).

6bis · Portabilidad — requisito, no aspiración ☐

El dueño ha fijado 2026-09-21: **la plataforma debe correr en varias máquinas Linux**, y Windows se quiere para desarrollo porque el servidor Linux le va lento.

Estado medido hoy:

mitad	módulos	Linux	Windows
export, autoría, curación	<code>core/worker.py</code> , <code>core/exportdrv.py</code> , <code>sqx/export/</code> , <code>sqx/curate/</code>	☐	☐ <code>require_posix()</code> en 8 puntos de entrada
análisis	<code>core/surface/</code> , <code>tasks/</code> , <code>strategies/</code> , <code>portfolio/</code> , informes	☐	☐

El único bloqueo es `bin/sqx-worker.sh`: bash, con `rsync`, `ss`, `curl`, `setsid`. Portarlo a Python multiplataforma borra la división entera. **Se hace en W3**, que es cuando esa capa se toca.

⚠ **Deuda adyacente que hay que arreglar en el mismo lote:** `bin/clone-sqx-worker.sh` y `bin/sqx-worker.sh` llevan las rutas de esta máquina **codificadas a fuego** (`/home/sergioguslw/Desktop/SQX`). `checks.py` solo inspecciona ficheros `.py` , así que no las ve. En una máquina nueva operan sobre rutas inexistentes — o peor, sobre el install equivocado. Deben leer de `config/machine.yaml` como todo lo demás.

Entregable de esta sección, ya escrito: `docs/SETUP-NEW-MACHINE.md` — el runbook para el agente que configure la plataforma en cada máquina: prerequisites, cuántas instalaciones de SQX y por qué, el clonado del worker con sus cinco parches y la verificación que impide que el worker sea un alias silencioso del maestro, la configuración del proyecto, las barras, el límite de Windows y la lista de verificación final.

7 · Reglas del proyecto que cruzan este trabajo

regla	dónde muerde
1 sync borra <code>.sqx</code> no en memoria	W3: guarda de conteo + snapshot
2 nunca <code>sqcli</code> contra el master con GUI; nunca <code>kill -f StrategyQuantX</code>	W3: worker 5060, un trabajo, <code>bin/sqx-worker.sh stop</code>
3 no arrancar builds ni cambiar qué construye un proyecto	el dueño corre los SPP; el proyecto de variantes es nuevo , no una modificación del suyo
4 nunca editar un <code>project.cfx</code> que una instancia viva tiene abierto	W3: el proyecto se crea en el worker y se entrega como <code>.cfx</code> para importar por GUI
5 <code>python3 -m core.assets <SYMBOL></code> antes de autorizar nada	bloquea W3 hoy: sale con código 2
8 comando nuevo = página de manual en español en la misma tarea	W1, W3, W4, W5, W6
9 <code>CODESTYLE.md</code> , luego <code>depmap.py</code> + <code>checks.py</code>	todos

Páginas de manual a crear (`docs/manual/` , copiando `_PLANTILLA.md` , en español, con capturas de salida real): `sppUltra` , `variantes` (W2+W3), `walkForwardCorrelation` , `walkForwardMatrix` , `pipeline` . `checks.py` falla si un `__main__` no aparece en ninguna página.

⚠ **La GUI del worker.** El binario existe (`~/Desktop/SQX_w1/StrategyQuantX`) y el dueño puede abrirla, pero **nunca a la vez que el demonio de 5060**, y abrirla dispara syncs — con un databank de 5.000 variantes dentro, es el escenario del log de USDJPY. Inspeccionar **antes** de fabricar o **después** de recoger, nunca en medio. (¶ inferido del comportamiento del master, sin verificar.)

8 · Verificación

W0 — `python3 -m pytest tests/` con las tres rejillas sintéticas. La barajada es la decisiva: si $HL\text{-}shift \approx 0$ pero $\rho \approx 0$, la librería ve lo que las distribuciones no ven.

W1 — `python3 -m strategies.sppUltra.report --project XAUUSD --databank "SPP IS"` sobre `raw/XAUUSD/SPP_IS/2026-09-10` (ya en disco). Comprobaciones contra lo medido: `CBlock_SqzMmnInt21` debe dar **757 grupos, 757 con NetProfit y nº de operaciones idénticos**; y `DICrossShift1` en `Strategy` `17.9.39` debe explicar **22,6 % de la varianza IS y 67,6 % de la OOS**.

- W2** — fabricar 20 variantes y verificar: 20 `tuple_hash` distintos, `<Fingerprint>` ausente en los 20, `variant_id` legible dentro del `.sqx`, `origin: true` en exactamente una fila.
- W3** — **la prueba de 3 ficheros primero**. Los canarios deben devolver `P00000` 9.076,34 / 844 ops, `P00001` 2.039,86 / 4 ops, `P00002` 3.077,74 / 6 ops. Si no salen idénticos, la fabricación está rota y se sabe **antes** de analizar nada. Después, 100 variantes para medir el throughput del export.
- W4/W5** — reconciliación del `UlcerPerformanceIndex`; y el informe reproduce $p \approx +0,31$ como referencia de la optimización secuencial.
- W6** — matar el proceso a mitad de las 5.000 y relanzarlo: debe continuar sin refabricar, y el `state.json` debe reflejar exactamente dónde estaba.
- Todos** — `python3 tools/depmap.py && python3 tools/checks.py` en verde, y la página de manual escrita en la misma tarea.
-

9 · Bloqueos y orden de arranque

	estado
<code>core.assets XAUUSD</code> sale con código 2	bloquea W3 ; W7 lo resuelve rellenando provisionalmente
Fases 1/2/3 del SPP sin correr	el dueño las lanza; W1 se escribe y testea con la Fase 0 que ya hay
Pre-registro del holdout	debe existir antes de la primera lectura de 2022–2026
Umbral del veredicto	PROPUESTOS; pendientes de aprobación del dueño
Topología de instalaciones SQX (¿2 o 3? ¿una máquina o varias?)	es la misma decisión que la pregunta abierta nº 2 del documento de plataforma, que dice que «decide la arquitectura entera y no se puede posponer»
Licencia de SQX ante multi-instancia headless	⚠ nadie ha leído el EULA. Riesgo 1 del documento de plataforma

Estado a 2026-09-21

lote	estado
W0 core/surface/ + tests	▯ hecho
W1 strategies/sppUltra/	▯ hecho — módulo, manual 12-sppultra.md , knowhow
W4 strategies/walkForwardMatrix/	▯ hecho — módulo, manual 14-walkforwardmatrix.md , knowhow
W7 presupuesto de disco + costes provisionales	▯ hecho — perf/disk/budget.py + retention.py , report sale non-zero al pasarse, assets/XAUUSD.yaml relleno y core.assets ya sale 0. Más docs/SETUP-NEW-MACHINE.md
W2 sqx/variants/ diseño y fabricación	▯ pausado por la topología de SQX
W3 sqx/variants/ ejecución y recogida	▯ bloqueado por costes + worker + \$6bis
W5 walkForwardCorrelation/	▯ necesita datos de W3
W6 pipeline/	▯
W8 multi-mercado \$9	▯
Las 6 skills + el orquestador	▯ ninguna escrita
Pre-registro del holdout	▯
UlcerPerformanceIndex	▯ reconciliado, y no reconcilia : forma confirmada ($\rho +0,998$ con AnnualPctReturn/UlcerIndex) pero la constante no converge y el residuo depende de NumberOfTrades y DataLength . Se une a los no reconstruibles; se exporta y se usa el de SQX

Siguiente sin bloqueos: W6. W2 y W3 esperan a la decisión de topología de SQX; W5 y W8 esperan a que W3 produzca datos.

10 · Lo que se escribe en knowhow/ al terminar

- 04-export.md — Parquet 6,9× vs poda 3,8×; los centinelas de RExpectancy ; las 42 constantes; los 11 grupos redundantes; la redundancia intra-ventana que desaparece entre ventanas; el sesgo $\sqrt{T}$ medido en las placebo; la asimetría del filtro de ≥ 100 operaciones; el throughput del export masivo.
- 01-file-formats.md — CalmarRatio? = CAGR/DrawdownPct e idéntica a AnnualPctReturnDDRatio? ; las columnas con ? literal; RExpectancyScore $\approx$ RExpectancy $\times\sqrt{n}$; Balance = capital + cumsum(P/L); Ticket = orden por (Open, Close); qué borrar del settings.xml para que el databank no deduplique; si el .sqx de 5 miembros carga.
- 03-driving-sqx.md — la guarda de conteo de databank contra el sync; si la GUI del worker es utilizable y con qué precauciones.

Apéndice — lo que NO se ha verificado

Por honestidad, y porque un dossier que se lee como certero es peor que ninguno: se construye encima.

Del protocolo original, y sigue abierto:

- **Si SQX carga un .sqx de 5 miembros**, sin `orders.bin` ni curva de equity. Decide si las 5.000 variantes ocupan 57 MB o 500 MB. Se mata con 3 ficheros y los canarios.
- **Si el databank deduplica por el <Fingerprint> heredado** del original. Misma prueba.
- **Cuánto tarda exportar los trades de 5.000 estrategias**. Nadie lo ha medido. Medir con 100 y extrapolar antes de comprometerse.
- **La forma exacta de la respuesta de -project action=status** en una tarea en marcha.

Del trabajo de esta sesión:

- **Las cifras de tamaño para 5.000 variantes son aritmética, no medición**. Los tamaños unitarios del `.sqx` (11,3 KB / 100 KB / 5.215 KB) y el throughput del retest (~24 estrategias/s) están medidos en el protocolo original; multiplicarlos por 5.000 es una extrapolación que asume que el coste es lineal y que nada se degrada con el tamaño del databank.
- **La GUI del worker**. El binario existe en `~/Desktop/SQX_w1/StrategyQuantX`, pero que abrirla dispare syncs está **inferido** del comportamiento documentado del maestro, no probado. Si importa, verificarlo con el worker vacío antes de que haya nada que perder.
- **Dos cifras del protocolo original no reproducen**. Los «757 grupos» de `CBlock_SqzMmnInt21` dan **217** en `Strategy 17.9.39` y **440** sumando las tres estrategias que llevan ese bloque; 757 es exactamente la cifra de la fila «MC Trades — 757 .sqx reteseados» del §6 del mismo documento, así que se sospecha transcripción. Y el «22,6 % de la varianza IS» de `DICrossShift1` sale 0,342 sobre el export pareado. El **67,6 % OOS sí reproduce: 0,6817**.
- **walkForwardMatrix concluye sobre dos estrategias**. Dos no son una población. `WFM_Stability` (1,5 GB, misma fecha) sigue sin leerse y es el input obvio para saber si `perverse` es propiedad de `Strategy 4.33.46` o de cómo se construyen estas estrategias.
- **El veredicto perverse tiene una explicación inocente que no se ha descartado**: un mercado que alterne regímenes al ritmo del tramo produciría el mismo resultado sin que la estrategia ni el procedimiento tengan nada malo. Haría falta una clasificación de regímenes o una estrategia de control.
- **Los umbrales del veredicto de WFC (§6, lote W5) son PROPUESTOS**, no aprobados por el dueño.
- **Los presupuestos de disco de W7** se dimensionaron con 2,3 GB en uso y una extrapolación del estudio de variantes. No hay histórico contra el que calibrarlos.
- **Los costes de assets/XAUUSD.yaml son los defaults de SQX**, no valores acordados con Infinox. Todo resultado que dependa de costes producido antes de que el dueño los sustituya lleva esa salvedad.
- **La licencia de SQX ante multi-instancia headless**. Nadie ha leído el EULA. Es el riesgo 1 de `plataforma-unificada-2026-09-20.md` y puede matar la parte de automatización.

Lo que sí está medido lleva `knowhow/`, que es donde sobrevive.